

Что ещё не сделано по 1.md

Документ составлен по ТЗ из 1.md и текущему состоянию проекта.

Шаг 1 - структура проекта

Текущую структуру проекта оставляем как есть:

```
app/client
app/server
```

Перенос в `apps/web`, `apps/api`, `packages/shared` не делаем.

Шаг 2 - frontend: закрыть требования по UI и библиотекам

Часть frontend уже есть: Vite, React, TypeScript, React Router, TanStack Query, Axios, SCSS Modules, страницы запусков, детали запуска, вкладки артефактов, логов и кода.

Сделано:

1. `React Markdown` уже установлен и подключен для `visual-report.md`.
2. `build-error.md` и `reference-validation.md` добавлены в отображение результата запуска.
3. Для удаления, скачивания кода и пересборки добавлены toast-уведомления.
4. Polling статуса запуска уже есть для статусов `queued` и `running`.
5. Frontend build успешно проходит.

Что ещё проверить вручную:

1. Убедиться в браузере, что UI на странице деталей показывает все обязательные артефакты из ТЗ:
 - reference image;
 - preview/result;
 - desktop screenshot;
 - mobile screenshot;
 - diff image;
 - markdown visual report;
 - кнопку скачивания frontend-кода.
2. Проверить toast-уведомления на реальном backend-запуске.

Шаг 3 - backend: вынести pipeline из одного сервиса

Сейчас большая часть pipeline находится в `RunsService`.

Что ещё не сделано:

1. Разнести backend по модулям из ТЗ:

- `runs;`
 - `generation;`
 - `storage;`
 - `screenshot;`
 - `visual-qa.`
2. Вынести файловое хранилище артефактов в отдельный сервис.
 3. Вынести генерацию `project spec/design/code` в отдельный `generation`-сервис.
 4. Вынести Playwright-скриншоты в `screenshot`-сервис.
 5. Вынести `pixel diff` и `visual report` в `visual-qa`-сервис.

Шаг 4 - очередь фоновых задач

Сейчас pipeline запускается напрямую через `void this.processRun(...)`.

Что ещё не сделано:

1. Подключить BullMQ или другой `job queue`.
2. Использовать Redis как `backend` очереди, если выбран BullMQ.
3. Добавить `producer` при создании запуска.
4. Добавить `worker`, который выполняет `pipeline`.
5. Обновлять статус запуска из `worker`.
6. Обработать `retry/fail` для фоновой задачи.

Шаг 5 - AI image generation для визуального референса

Сейчас референс создается программно как SVG.

Что ещё не сделано:

1. Подключить AI image generation API.
2. Сформировать `prompt` для генерации одного `hero`-блока.
3. Сохранять результат как:

```
generated/runs/run-XXX/reference/hero-reference.png
```

4. Соблюдать требования из ТЗ:
 - один `hero`-блок;
 - без браузерной рамки;
 - без лишних секций;
 - примерно `1440x900`;
 - реализуемо через `frontend`;
 - без сложных 3D;
 - читаемый текст;
 - понятная структура.
5. Добавить конфигурацию API ключей через `.env`.

Шаг 6 - проверка визуального референса

Сейчас отдельной проверки качества `reference image` нет.

Что ещё не сделано:

1. Проверять референс до генерации кода.
2. Валидировать:
 - понятность сетки;
 - читаемость текста;
 - расположение элементов;
 - отсутствие слишком сложной графики;
 - реализуемость через CSS/Tailwind;
 - отсутствие лишних блоков;
 - соответствие брифу.
3. Если референс плохой, создавать:

```
generated/runs/run-XXX/qa/reference-validation.md
```

4. Ставить статус `reference_failed`.
5. Останавливать pipeline при плохом референсе.

Шаг 7 - AI vision/text model для описания дизайна

Сейчас `design-description.md` и `design-tokens.json` создаются шаблонно из брифа.

Что ещё не сделано:

1. Подключить AI vision/text model для анализа `reference image`.
2. Генерировать `design-description.md` на основе изображения, а не только брифа.
3. Генерировать `design-tokens.json` на основе изображения, а не только шаблона.
4. Проверить, что описание содержит:
 - фон;
 - сетку;
 - типографику;
 - кнопки;
 - карточки;
 - декоративные элементы;
 - адаптивное поведение.

Шаг 8 - генерация frontend-кода по ТЗ

Сейчас генерируется минимальный `Vite + React + TypeScript` проект в папке `code`, без Next.js и Tailwind.

Что ещё не сделано:

1. Генерировать проект в пути:

generated/runs/run-XXX/code/frontend-project/

2. Использовать стек из ТЗ:

- Next.js;
- React;
- TypeScript;
- Tailwind CSS.

3. Создавать минимальную структуру:

```
frontend-project/  
  app/  
    page.tsx  
    layout.tsx  
    globals.css  
  components/  
    Hero.tsx  
  public/  
  package.json  
  tailwind.config.ts  
  tsconfig.json
```

4. Генерировать код через AI code generation, если это остается обязательным требованием.

5. Проверять, что сгенерированный сайт содержит:

- одну страницу;
- один hero-блок;
- текст из брифа;
- композицию по reference image;
- без сторонних UI-библиотек;
- без лишних секций.

Шаг 9 - build и технический repair

Build уже запускается, зависимости устанавливаются, максимум 2 попытки частично реализованы.

Что ещё не сделано:

1. Адаптировать build под новый путь `code/frontend-project`.
2. Адаптировать команды под Next.js.
3. Реализовать исправление только технических ошибок, а не просто повторную попытку той же сборки.
4. Сохранять полный полезный контекст ошибки в `qa/build-error.md`.
5. Проверить корректную установку статуса `build_failed`.

Шаг 10 - preview и скриншоты

Скриншоты через Playwright уже есть, но привязаны к Vite preview и фиксированному порту.

Что ещё не сделано:

1. Адаптировать preview/dev server под Next.js.
2. Избежать конфликта фиксированного порта **4173** при параллельных запусках.
3. Гарантированно дожидаться готовности preview-сервера перед скриншотами.
4. Сохранять скриншоты в требуемые пути:

```
generated/runs/run-XXX/screenshots/rendered-desktop.png
generated/runs/run-XXX/screenshots/rendered-mobile.png
```

5. Проверить размеры:
 - desktop **1440x900**;
 - mobile **390x844**.

Шаг 11 - визуальное сравнение и отчет

Pixel diff уже есть, **diff.png** и **visual-report.md** создаются. Но отчет сейчас шаблонный и содержит ошибку формата оценки: **score** считается как 0-100, но выводится как **/10**.

Что ещё не сделано:

1. Исправить формат оценки в **visual-report.md**.
2. Сделать отчет ближе к T3:
 - общая оценка **7/10** или аналогичная шкала;
 - что совпало;
 - что не совпало;
 - критичные проблемы;
 - рекомендации для исправления.
3. Добавить смысловую visual QA проверку, а не только pixelmatch.
4. Сравнить именно **hero-reference.png** с **rendered-desktop.png** после перехода на PNG-reference.
5. Корректно ставить **visual_failed** или **needs_manual_review**, если качество ниже порога.

Шаг 12 - финальные статусы

Часть статусов уже есть.

Что ещё не сделано:

1. Проверить наличие всех финальных статусов из T3:
 - **completed**;
 - **reference_failed**;
 - **build_failed**;
 - **visual_failed**;
 - **needs_manual_review**;
 - **failed**.
2. Убедиться, что **status.json** обновляется на каждом важном шаге.

3. Добавить `score` и ошибку в `status.json`, если это нужно UI.
4. Проверить, что финальный статус всегда выставляется при ошибках pipeline.

Шаг 13 - API и безопасность артефактов

Базовые endpoints для запусков, артефактов и скачивания кода уже есть.

Что ещё не сделано:

1. Проверить валидацию доступа ко всем артефактам.
2. Проверить безопасную работу с путями файлов.
3. Добавить ограничения на размер/тип скачиваемых и читаемых файлов, если нужно.
4. Убедиться, что ZIP скачивает именно `frontend-project`, а не служебные файлы.

Шаг 14 - Docker Compose и локальный запуск

Docker Compose есть, но нужно проверить соответствие фактическому backend.

Что ещё не сделано:

1. Проверить PostgreSQL в `docker-compose.yml`.
2. Проверить Redis в `docker-compose.yml` после добавления BullMQ.
3. Проверить `.env.example` на все нужные переменные:
 - database;
 - redis;
 - generated root;
 - AI API keys;
 - frontend/backend URLs.
4. Обновить README с точными командами запуска.

Шаг 15 - критерии готовности MVP

По критериям из `1.md` нужно финально проверить:

1. Приложение запускается локально через README.
2. Frontend открывается в браузере.
3. Пользователь может отправить бриф.
4. Backend создает `Run` в PostgreSQL.
5. Backend выполняет pipeline в фоне через очередь.
6. Все артефакты сохраняются в `generated/runs/run-XXX/`.
7. UI показывает статус запуска.
8. UI показывает reference image, screenshots, diff и markdown-отчет.
9. Сгенерированный frontend-код можно скачать.
10. Ошибки build/reference validation сохраняются в отчетах.

Не делать сейчас

Эти пункты из `1.md` не входят в MVP и пока не должны браться в работу:

1. Регистрация.
2. Авторизация.
3. Роли пользователей.
4. Оплата.
5. Личный кабинет.
6. История проектов между пользователями.
7. Drag-and-drop редактор.
8. Figma export.
9. Deploy на Vercel.
10. Генерация полного сайта.
11. Многостраничные сайты.
12. CMS.
13. Формы.
14. Ручная адаптивная кастомизация.
15. Красивый сложный UI.
16. WebSocket, если хватает polling.
17. Автоматический visual repair loop.

Рекомендуемый порядок выполнения дальше

1. Исправить мелкие несоответствия UI: Markdown-render, ошибки артефактов, polling.
2. Исправить visual report: шкала оценки, структура отчета, статусы качества.
3. Перенести сгенерированный проект на **Next.js + Tailwind** и путь **code/frontend-project**.
4. Адаптировать build/preview/screenshots под Next.js.
5. Добавить проверку reference image и статус **reference_failed**.
6. Вынести pipeline в отдельные backend-сервисы.
7. Подключить очередь BullMQ + Redis.
8. Подключить AI image generation и AI vision/text/code generation.
9. Обновить Docker Compose, **.env.example** и README.
10. Пройти финальный checklist MVP.